

C# - Phần 10.3: LINQ First(), FirstOrDefault(), Single(), SingleOrDefault()

Lý thuyết - Câu hỏi lý thuyết - Bài tập - Đáp án - Lưu ý

Mục tiêu: Hiểu cách lấy một phần tử bằng LINQ, phân biệt First/FirstOrDefault với Single/SingleOrDefault, hiểu trường hợp trả default, trường hợp throw exception, và biết khi nào kết quả không cần foreach.

PHẦN A - LÝ THUYẾT

1. Nhóm method lấy một phần tử

Bốn method First(), FirstOrDefault(), Single(), SingleOrDefault() đều dùng khi kết quả cuối cùng cần là một phần tử thay vì một collection.

```
First()
FirstOrDefault()
Single()
SingleOrDefault()
```

Vì kết quả cuối cùng là một object hoặc một giá trị đơn, thông thường không cần foreach để lấy nó ra.

2. First()

First() lấy phần tử đầu tiên của collection, hoặc phần tử đầu tiên thỏa điều kiện Lambda.

```
Product firstProduct =
    products.First();

Product expensiveProduct =
    products.First(
        product => product.Price >= 1000);
```

Nếu có nhiều phần tử thỏa điều kiện, First() chỉ lấy phần tử đầu tiên.

Nếu không tìm thấy: First() throw InvalidOperationException.

3. FirstOrDefault()

FirstOrDefault() giống First(), nhưng nếu không có phần tử phù hợp thì trả giá trị mặc định thay vì throw.

```
Product? product =
    products.FirstOrDefault(
        product => product.Id == "P99");
```

Với reference type như Product, giá trị mặc định là null.

```
if (product == null)
{
    Console.WriteLine("Product not found.");
}
else
{
    Console.WriteLine(product);
}
```

4. Vì sao thường dùng Product? với FirstOrDefault()

Dấu ? cho biết biến có thể nhận null. Đây là hợp lý vì FirstOrDefault() có thể không tìm thấy phần tử.

```
Product? product = products.FirstOrDefault(...);
```

Phân biệt: First() và Single() không trả null khi không tìm thấy; chúng throw. Vì vậy thường không cần khai báo Product? cho hai method này.

5. Single()

Single() yêu cầu có chính xác một phần tử phù hợp.

```
Product product =
    products.Single(
        product => product.Id == "P02");
```

Single() sẽ throw InvalidOperationException nếu không có phần tử phù hợp hoặc nếu có nhiều hơn một phần tử phù hợp.

Điểm quan trọng: Single() không kiểm tra các object có 'trùng nhau' hay không. Nó kiểm tra số lượng phần tử thỏa điều kiện phải đúng bằng 1.

6. SingleOrDefault()

SingleOrDefault() có ba trường hợp:

```
0 phần tử phù hợp
→ default

1 phần tử phù hợp
→ trả phần tử đó

>1 phần tử phù hợp
→ throw InvalidOperationException
```

Với Product, default là null.

7. Bảng so sánh bốn method

Method	0 phần tử	1 phần tử	>1 phần tử
First()	Throw	Trả phần tử	Trả phần tử đầu tiên
FirstOrDefault()	Default	Trả phần tử	Trả phần tử đầu tiên
Single()	Throw	Trả phần tử	Throw
SingleOrDefault()	Default	Trả phần tử	Throw

8. First và Single khác nhau ở bản chất nào?

```
First
→ "Cho tôi phần tử đầu tiên phù hợp."

Single
→ "Tôi kỳ vọng có đúng một phần tử phù hợp."
```

First không quan tâm có bao nhiêu phần tử phù hợp. Single dùng khi nghiệp vụ kỳ vọng tính duy nhất.

9. OrDefault có nghĩa gì?

Từ Default có nghĩa là giá trị mặc định của kiểu dữ liệu.

```
default(Product) → null
default(string) → null
default(int) → 0
default(decimal) → 0m
default(bool) → false
```

Lưu ý: Với value type như int, FirstOrDefault() không tìm thấy sẽ trả 0, nên đôi khi khó phân biệt 0 là dữ liệu thật hay default.

10. FirstOrDefault() với Where()

```
Product? product =
    products
        .Where(
            product => product.Price >= 1000)
        .FirstOrDefault();
```

Where() trả một chuỗi nhiều Product, sau đó FirstOrDefault() lấy một Product đầu tiên.

Có thể viết ngắn hơn:

```
Product? product =
    products.FirstOrDefault(
        product => product.Price >= 1000);
```

11. Where + Select + FirstOrDefault()

```
string? name =
    products
        .Where(
            product => product.Price >= 1000)
        .Select(
            product => product.Name)
        .FirstOrDefault();

List<Product>
    ↓ Where
IEnumerable<Product>
    ↓ Select
IEnumerable<string>
```

```
↓ FirstOrDefault  
string?
```

Kết quả cuối cùng là một string?, không phải collection, nên không cần foreach.

12. Khi nào không cần foreach?

Nếu truy vấn cuối cùng trả một object hoặc một giá trị đơn, ta có thể dùng trực tiếp.

```
Product? product =  
    products.FirstOrDefault(  
        p => p.Id == "P01");  
  
Console.WriteLine(product);
```

Ghi nhớ: foreach thường dùng khi muốn duyệt nhiều phần tử. First/FirstOrDefault/Single/SingleOrDefault trả một phần tử nên thường không cần foreach.

PHẦN B - CÂU HỎI LÝ THUYẾT

1. First() dùng để làm gì?
2. First() và FirstOrDefault() khác nhau thế nào?
3. Single() và First() khác nhau thế nào?
4. SingleOrDefault() xử lý 0, 1 và nhiều hơn 1 phần tử ra sao?
5. Vì sao kết quả FirstOrDefault() với Product thường khai báo là Product??
6. Khi nào nên dùng Single() thay vì First()?
7. Với products.Where(...).Select(...).FirstOrDefault(), hãy mô tả kiểu dữ liệu thay đổi qua từng bước.
8. Vì sao kết quả cuối cùng của truy vấn trên không cần foreach?

PHẦN C - BÀI TẬP 10.3

Tiếp tục sử dụng class Product của bài trước và dữ liệu:

```
List<Product> products =  
    new List<Product>  
    {  
        new Product("P01", "Mouse", 300),  
        new Product("P02", "Keyboard", 700),  
        new Product("P03", "Monitor", 2500),  
        new Product("P04", "Laptop", 15000),  
        new Product("P05", "USB", 150)  
    };
```

- Dùng First() lấy Product đầu tiên.
- Dùng First() lấy Product đầu tiên có Price >= 1000.
- Dùng FirstOrDefault() tìm Product có Id == "P99" và kiểm tra null.
- Dùng Single() tìm Product có Id == "P02".
- Dùng SingleOrDefault() tìm Product có Id == "P99".
- Test Single() với điều kiện product => product.Price >= 1000 và dùng try/catch để quan sát exception.
- Dùng Where + Select + FirstOrDefault để lấy Name đầu tiên của Product có Price >= 1000.
- Hiển thị các kết quả phù hợp mà không dùng foreach cho các truy vấn trả về một phần tử.

PHẦN D - ĐÁP ÁN CODE

```
public class Program  
{  
    public static void Main(string[] args)  
    {  
        List<Product> products =  
            new List<Product>  
            {  
                new Product("P01", "Mouse", 300),  
                new Product("P02", "Keyboard", 700),  
                new Product("P03", "Monitor", 2500),  
                new Product("P04", "Laptop", 15000),  
                new Product("P05", "USB", 150)  
            };  
    }  
};
```

```

// 1. First Product
Product product1 =
    products.First();

Console.WriteLine(product1);
Console.WriteLine("-----");

// 2. First Product with Price >= 1000
Product product2 =
    products.First(
        product => product.Price >= 1000);

Console.WriteLine(product2);
Console.WriteLine("-----");

// 3. FirstOrDefault - P99
Product? product3 =
    products.FirstOrDefault(
        product => product.Id == "P99");

if (product3 == null)
{
    Console.WriteLine("Product not found.");
}
else
{
    Console.WriteLine(product3);
}

Console.WriteLine("-----");

// 4. Single - P02
Product product4 =
    products.Single(
        product => product.Id == "P02");

Console.WriteLine(product4);
Console.WriteLine("-----");

// 5. SingleOrDefault - P99
Product? product5 =
    products.SingleOrDefault(
        product => product.Id == "P99");

if (product5 == null)
{
    Console.WriteLine("Product not found.");
}
else
{
    Console.WriteLine(product5);
}

Console.WriteLine("-----");

// 6. Single with more than one match
try
{
    Product product6 =
        products.Single(
            product => product.Price >= 1000);

    Console.WriteLine(product6);
}
catch (InvalidOperationException e)
{
    Console.WriteLine(e.Message);
}

Console.WriteLine("-----");

// 7. Where + Select + FirstOrDefault
string? product7 =
    products
        .Where(
            product => product.Price >= 1000)

```

```

        .Select(
            product => product.Name)
        .FirstOrDefault();

    if (product7 == null)
    {
        Console.WriteLine("Product not found.");
    }
    else
    {
        Console.WriteLine(product7);
    }
}
}

```

PHẦN E - ĐÁP ÁN CÂU HỎI LÝ THUYẾT

1. First() dùng để làm gì?

First() lấy phần tử đầu tiên của collection hoặc phần tử đầu tiên thỏa điều kiện. Nếu không tìm thấy, nó throw `InvalidOperationException`.

2. First() và FirstOrDefault() khác nhau thế nào?

Cả hai đều lấy phần tử đầu tiên phù hợp. First() throw nếu không tìm thấy; FirstOrDefault() trả default.

3. Single() và First() khác nhau thế nào?

First() chỉ cần phần tử đầu tiên phù hợp. Single() yêu cầu phải có chính xác một phần tử phù hợp; 0 hoặc nhiều hơn 1 đều throw.

4. SingleOrDefault() xử lý thế nào?

0 phần tử -> default; 1 phần tử -> trả phần tử đó; hơn 1 phần tử -> throw `InvalidOperationException`.

5. Vì sao thường dùng Product??

Vì FirstOrDefault() hoặc SingleOrDefault() có thể trả null khi không tìm thấy Product.

6. Khi nào dùng Single()?

Khi nghiệp vụ kỳ vọng dữ liệu phải duy nhất, ví dụ tìm theo một Id được thiết kế là duy nhất.

7. Kiểu dữ liệu thay đổi qua Where + Select + FirstOrDefault?

List<Product> -> IEnumerable<Product> sau Where -> IEnumerable<string> sau Select Name -> string? sau FirstOrDefault.

8. Vì sao không cần foreach?

Vì FirstOrDefault() đã trả một giá trị duy nhất, ví dụ string?, thay vì một collection như IEnumerable<string>.

PHẦN F - ĐÁNH GIÁ BÀI LÀM VÀ LƯU Ý

- Bài làm đạt khoảng 9.3/10.
- First() lấy phần tử đầu tiên: đúng.
- First() với Price >= 1000: đúng; kết quả là Monitor.
- FirstOrDefault() tìm P99 và kiểm tra null: đúng.
- Single() tìm P02: đúng.
- SingleOrDefault() tìm P99: đúng.
- Single() với Price >= 1000 throw vì có Monitor và Laptop cùng thỏa điều kiện.
- Where + Select + FirstOrDefault trả Monitor và không cần foreach: đúng.
- Nên bắt `InvalidOperationException` thay vì catch(Exception) khi biết chính xác lỗi có thể xảy ra.

1. First()/Single() thường không cần Product?

Vì hai method này không trả null khi không tìm thấy; chúng throw. Do đó có thể khai báo Product thay vì Product?.

```

Product product1 = products.First();

Product product2 =
    products.Single(
        p => p.Id == "P02");

```

2. Single() không có nghĩa là 'không được trùng'

Cách nói chính xác là Single() yêu cầu có đúng một phần tử thỏa điều kiện.

```
products.Single(  
    p => p.Price >= 1000);
```

Monitor và Laptop là hai object khác nhau nhưng cùng thỏa điều kiện, nên Single() vẫn throw.

3. Tập phản xạ theo đổi kiểu sau mỗi LINQ method

```
products  
→ List<Product>  
  
.Where(...)  
→ IEnumerable<Product>  
  
.Select(p => p.Name)  
→ IEnumerable<string>  
  
.FirstOrDefault()  
→ string?
```

Mục tiêu: Khi đọc chuỗi LINQ, ngoài việc hiểu nghiệp vụ, hãy tập xác định kiểu dữ liệu sau mỗi bước.

4. Tổng kết ngắn

```
First  
→ lấy phần tử đầu tiên  
→ 0 phần tử: throw  
  
FirstOrDefault  
→ lấy phần tử đầu tiên  
→ 0 phần tử: default  
  
Single  
→ yêu cầu đúng 1 phần tử  
→ 0 hoặc >1: throw  
  
SingleOrDefault  
→ 0: default  
→ 1: trả phần tử  
→ >1: throw
```

Câu nối bài sau: Sau khi đã biết lọc, biến đổi và lấy một phần tử, phần tiếp theo của LINQ sẽ tiếp tục mở rộng các thao tác truy vấn trên collection.